

How the GRPO Training Loss Is Defined, How It's Actually Computed, and Why the Numerical Value Sits Near Zero

1. Loss definition

For a single trajectory i inside group g (sampled with the current policy π_θ), the policy-gradient loss is

$$L_{g,i} = -A_{g,i} \cdot \overline{\log p_\theta}(\text{actions of trajectory } i) \quad (1)$$

First factor — the advantage $A_{g,i}$ — is the within-group z -score of the terminal reward:

$$A_{g,i} = \frac{r_{g,i} - \bar{r}_g}{\sigma_{r,g} + \varepsilon} \quad \text{where } \bar{r}_g = \frac{1}{G} \sum_j r_{g,j}, \quad \sigma_{r,g} = \sqrt{\frac{1}{G} \sum_j (r_{g,j} - \bar{r}_g)^2}. \quad (2)$$

In words: “this trajectory’s terminal reward, minus the average reward of the eight trajectories in its group, divided by the within-group reward standard deviation.”

Second factor — the mean log-probability $\overline{\log p_\theta}$ — is the policy’s log-probability of the trajectory’s actually-taken action tokens, averaged over those tokens:

$$\overline{\log p_\theta} = \frac{1}{T} \sum_{t=1}^T \log p_\theta(\text{token}_t \mid \text{prefix}_t). \quad (3)$$

In words: “during the rollout you actually emitted tokens $\text{token}_1, \dots, \text{token}_T$. Now run the model on the re-tokenized chat and ask: what log-probability does the (current) policy give to each of those exact tokens? Take the average.”

Total loss is the average of $L_{g,i}$ over all trajectory items in the batch:

$$L_{\text{total}} = \frac{1}{n_{\text{items}}} \sum_{g,i} L_{g,i}. \quad (4)$$

2. How it is actually computed (in code)

See `scripts/17_grpo_static.py`. The compute path for one item is:

1. Concatenate `prefix` (system+user history) and the trajectory’s `assistant_text`, run a forward pass through the model:

```
out = model(input_ids=enc["input_ids"], attention_mask=...)
logits = out.logits[:, :-1, :] # shape (1, T-1, V), V = 152K vocab
```

2. For every token position, compute $-\log p_\theta(\text{label}_t)$ via chunked cross-entropy (chunked along the time axis to avoid materializing the full $(T \times V)$ log-softmax tensor):

```
nll = F.cross_entropy(logits, labels, reduction="none")
# nll[t] = - log p_theta(token_t | prefix_t)
```

3. Mask out the prefix region, average over the assistant region:

```
mean_logp = -(nll * assistant_mask).sum() / assistant_mask.sum()
```

4. Form the per-item PG loss and *immediately* call `.backward()` (must not accumulate $L_{\text{total}} += L_b$ across items — that holds every forward pass’s autograd graph in memory and OOMs):

```
loss_i = -(advantage * mean_logp) / n_items
loss_i.backward()
```

After the loop completes, call `optimizer.step()` once.

3. Why the loss numerically sits near zero

The cleanest way to see this is with concrete numbers. Take a single group of $G = 8$ trajectories with terminal rewards

$$\mathbf{r} = [1, 1, 0, 0, 1, 0, 1, 0].$$

Then

$$\bar{r} = 0.5, \tag{5}$$

$$\sigma_r = 0.5, \tag{6}$$

$$A_i = \frac{r_i - 0.5}{0.5} = [+1, +1, -1, -1, +1, -1, +1, -1]. \tag{7}$$

Crucial observation: $\sum_i A_i = 0$. This is forced by the z -score normalization (the numerator subtracts the mean, so the standardized values sum to zero by construction).

For a freshly initialized 7B policy, the per-token log-probability of short “Thought...\nAction:..” continuations is around -3 (a vocabulary of 152K and slightly biased token frequencies). Suppose the eight trajectories happen to have slightly different mean log-probs:

$$\overline{\log p_{\theta}} \approx [-3.1, -2.9, -3.0, -3.2, -2.8, -3.1, -2.9, -3.0].$$

Then $\text{loss}_i = -A_i \cdot \overline{\log p_{\theta_i}}$:

i	A_i	$\overline{\log p_{\theta_i}}$	loss_i
1	+1	-3.1	+3.1
2	+1	-2.9	+2.9
3	-1	-3.0	-3.0
4	-1	-3.2	-3.2
5	+1	-2.8	+2.8
6	-1	-3.1	-3.1
7	+1	-2.9	+2.9
8	-1	-3.0	-3.0
$\sum_i$			-0.6
$\div n_{\text{items}} = 64$			≈ -0.009

This number is exactly in the range we observed empirically ($[-0.01, +0.06]$ across 20 training steps).

4. The intuition

Each individual term loss_i has magnitude ~ 3 (because $|A_i| \approx 1$ and $|\overline{\log p_{\theta_i}}| \approx 3$). **But $\sum_i A_i = 0$ kills the bulk of those ± 3 contributions almost completely.**

What survives is the correlation between A_i and the *deviation* of $\overline{\log p_{\theta_i}}$ from its group mean:

$$L_{\text{total}} \approx \sum_i A_i \cdot \left(\overline{\log p_{\theta_i}} - \text{mean}(\overline{\log p_{\theta}}) \right). \quad (8)$$

That correlation is the genuine training signal — and it is small in magnitude precisely because the policy at any given moment doesn’t yet distinguish good from bad trajectories all that strongly. Hence $L_{\text{total}} \sim 0.01$ in absolute value. **This is not a bug; it is the definitional behaviour of GRPO.**

5. Why the loss is not expected to monotonically decrease

- **Supervised-learning loss** measures “how wrong is the prediction” — it is bounded below by zero and decreases monotonically as the model learns.
- **Policy-gradient loss** (REINFORCE / GRPO) is constructed so that its *gradient* is an unbiased estimator of $\nabla J(\theta)$, the expected reward gradient. Its *value* is just an artefact: by the group-centering $\sum_i A_i = 0$, the loss has expected value 0. It oscillates around 0 regardless of whether the policy is improving.

A useful analogy: a supervised loss is like “how far you are from the finish line” — decreasing means you are getting closer. A policy-gradient loss is like “which way is the wind blowing” — the direction carries information, but the wind *speed* does not have to fall to zero as you sail.

6. What you should look at instead of the loss

To check whether GRPO training is doing useful work, the right diagnostics are:

1. **LoRA gradient L^2 norm trend**: should slowly decay as the policy becomes more confident in its preferred actions. Across our 20 steps it bounces in $[0.05, 0.20]$ without a clear trend — meaning “the trainer is producing real gradients” but “too few steps to see convergence dynamics.”
2. **Held-out task reward / success rate**: roll out the *updated* model on a held-out task suite and measure win rate. **We have not done this yet**, so any claim that the model is “getting better at ALFWorld” is currently unsupported.

7. What the experiment as it stands does and does not show

Claim	Evidence
The loss is computed correctly (forward, backward, optimizer all run)	✓
The gradient signal magnitude is preserved (baseline 0.086, gated 0.098)	✓
The gate saves 32.2% wall-clock per training step	✓
The model becomes better at ALFWorld	<i>not measured</i>
The training run is converged	<i>no</i> (20 steps; loss is also not the right